

Volume 06 Specifications - Marketplace Opportunity Automation Specifications

Version v31 draft

README.md

Marketplace Opportunity Automation Specifications

Version: v31 draft

Status: Draft

Document ID: MAL-V06-V31-README

Purpose

This release folder contains the official v31 documentation set for Marketplace Opportunity Automation Specifications.

Scope

- Covers Marketplace.
- Covers Opportunity Intelligence.
- Covers Freelance Automation.
- Covers Remote Work Automation.

Acceptance

- All documents contain implementation-level guidance.
- The release PDF and ZIP are generated and verified.
- MANIFEST and RELEASE_NOTES are updated for v31.

Marketplace Opportunity Automation Specifications Index

- [spec-06-31-01-marketplace.md](#) - Marketplace
- [spec-06-31-02-opportunity-intelligence.md](#) - Opportunity Intelligence
- [spec-06-31-03-freelance-automation.md](#) - Freelance Automation
- [spec-06-31-04-remote-work-automation.md](#) - Remote Work Automation

Marketplace Specification

Title: Marketplace Specification

Version: v31 draft

Status: Draft

Specification ID: MAL-V06-MARKETPLACE-SPEC-01

Related Blueprint Documents

- Volume 03 Master Blueprint
- Volume 04 Canonical Dictionary
- Volume 05 Engineering Standards
- Prior Volume 06 Specifications

Purpose

Define implementation-level requirements for Marketplace in Mariam AI Enterprise OS so engineering teams can build, test, operate, and govern it consistently.

Scope

- Covers system responsibilities, runtime behavior, interfaces, events, storage, security, observability, and acceptance evidence.
- Includes interactions with Enterprise Core, AI Society, Runtime Ecosystem, Knowledge System, Capability System, Governance, and Infrastructure.
- Excludes application-specific code, vendor credentials, and temporary implementation shortcuts.

Responsibilities

- Maintain authoritative state and lifecycle rules for Marketplace.
- Validate inputs, permissions, dependencies, and policy constraints before execution.
- Emit auditable events for lifecycle transitions, failures, recovery, and acceptance.
- Provide deterministic behavior that can be tested and traced across releases.

Functional Requirements

- SHALL expose versioned public interfaces for create, read, update, execute, validate, and audit operations.
- SHALL validate all inbound requests against schema, identity, policy, and dependency rules.
- SHALL support idempotency keys for commands that may be retried.
- SHALL publish events for accepted, rejected, started, completed, failed, recovered, and archived states.
- SHALL produce evidence bundles for review, compliance, and release acceptance.

Non-Functional Requirements

- MUST preserve tenant isolation, least privilege, and traceable authorization.
- MUST support observable latency, throughput, failure rate, queue depth, and recovery metrics.

- MUST recover safely after process restart, duplicate event delivery, or partial storage failure.
- SHOULD degrade gracefully when optional providers, plugins, connectors, or models are unavailable.

Inputs

- User, agent, workflow, runtime, provider, plugin, connector, and governance requests.
- Configuration, policies, feature flags, schemas, secrets references, and dependency metadata.
- Runtime events, task graph state, knowledge records, capability scores, and audit context.

Outputs

- Versioned records, execution results, validation reports, events, metrics, traces, logs, and acceptance evidence.
- Human-readable status summaries for dashboards, operations, release notes, and incident review.

Public Interfaces

- `create_marketplace(request)`
- `get_marketplace_state(id)`
- `validate_marketplace(payload)`
- `execute_marketplace(command)`
- `audit_marketplace(scope)`

Internal Interfaces

- Event Bus for durable event publication and subscription.
- Service Container for dependency injection and lifecycle management.
- Runtime Manager for execution dispatch and cancellation.
- Storage layer for records, snapshots, indexes, and evidence.
- Governance service for approval, policy, risk, and compliance checks.

Events

- `marketplace.created`
- `marketplace.validated`
- `marketplace.started`
- `marketplace.completed`
- `marketplace.failed`
- `marketplace.recovered`
- `marketplace.accepted`

Storage Requirements

- Store canonical records with identifiers, owner, version, state, timestamps, policy version, and trace ID.
- Store immutable event history and mutable read models separately where practical.
- Store evidence, validation reports, and recovery metadata with retention rules.

Security Requirements

- Enforce authentication, authorization, tenant isolation, and field-level redaction.
- Protect secrets by reference only; never persist raw credentials in logs, events, Markdown, or exports.
- Require approval for privileged, destructive, externally visible, financial, or compliance-sensitive actions.

Observability Requirements

- Emit structured logs with correlation IDs and decision reasons.
- Emit metrics for request count, success rate, failure rate, latency, retries, recovery success, and acceptance failures.
- Provide traces across public interface, policy validation, runtime execution, storage, events, and callbacks.

Failure Modes

- Invalid input, missing dependency, expired permission, unavailable runtime, conflicting state, duplicate command, or failed downstream provider.
- Incomplete event delivery, partial persistence, timeout, stalled execution, or acceptance failure.

Recovery Rules

- Retry idempotent commands with bounded backoff.
- Rebuild read models from event history when projections drift.
- Escalate to governance when automatic recovery changes risk, scope, budget, deadline, or external side effects.
- Preserve failed attempts and recovery decisions as audit evidence.

Test Requirements

- Unit tests for schemas, state transitions, permission checks, and event payloads.
- Integration tests for public interfaces, storage, event bus, runtime dependencies, and governance approvals.
- Failure tests for retries, duplicate events, unavailable dependencies, and recovery paths.
- Acceptance tests proving traceability from requirement to evidence.

Acceptance Criteria

- The specification is complete enough to create implementation tickets without hidden architectural decisions.
- Interfaces, events, storage, security, observability, failures, recovery, and tests are documented.
- The document traces to Blueprint, Standards, and release artifacts.

Implementation Notes

- Prefer small versioned interfaces and append-only event history.
- Keep provider-specific details behind adapters.
- Treat auditability and recovery as first-class design constraints.
- Validate schemas before work reaches runtime execution.

Traceability

Source	Relationship
MAL-V06-MARKETPLACE-SPEC-01	Defines v31 implementation requirements for Marketplace.

Source	Relationship
Volume 03	Blueprint source.
Volume 05	Engineering standards source.
RELEASE_NOTES_v31_draft.md	Release evidence.

Version History

Version	Date	Status	Notes
v31 draft	2026-06-29	Draft	Initial specification for Marketplace.

Opportunity Intelligence Specification

Title: Opportunity Intelligence Specification

Version: v31 draft

Status: Draft

Specification ID: MAL-V06-OPPORTUNITY-INTELLIGENCE-SPEC-02

Related Blueprint Documents

- Volume 03 Master Blueprint
- Volume 04 Canonical Dictionary
- Volume 05 Engineering Standards
- Prior Volume 06 Specifications

Purpose

Define implementation-level requirements for Opportunity Intelligence in Mariam AI Enterprise OS so engineering teams can build, test, operate, and govern it consistently.

Scope

- Covers system responsibilities, runtime behavior, interfaces, events, storage, security, observability, and acceptance evidence.
- Includes interactions with Enterprise Core, AI Society, Runtime Ecosystem, Knowledge System, Capability System, Governance, and Infrastructure.
- Excludes application-specific code, vendor credentials, and temporary implementation shortcuts.

Responsibilities

- Maintain authoritative state and lifecycle rules for Opportunity Intelligence.
- Validate inputs, permissions, dependencies, and policy constraints before execution.
- Emit auditable events for lifecycle transitions, failures, recovery, and acceptance.
- Provide deterministic behavior that can be tested and traced across releases.

Functional Requirements

- SHALL expose versioned public interfaces for create, read, update, execute, validate, and audit operations.
- SHALL validate all inbound requests against schema, identity, policy, and dependency rules.
- SHALL support idempotency keys for commands that may be retried.
- SHALL publish events for accepted, rejected, started, completed, failed, recovered, and archived states.
- SHALL produce evidence bundles for review, compliance, and release acceptance.

Non-Functional Requirements

- MUST preserve tenant isolation, least privilege, and traceable authorization.
- MUST support observable latency, throughput, failure rate, queue depth, and recovery metrics.

- MUST recover safely after process restart, duplicate event delivery, or partial storage failure.
- SHOULD degrade gracefully when optional providers, plugins, connectors, or models are unavailable.

Inputs

- User, agent, workflow, runtime, provider, plugin, connector, and governance requests.
- Configuration, policies, feature flags, schemas, secrets references, and dependency metadata.
- Runtime events, task graph state, knowledge records, capability scores, and audit context.

Outputs

- Versioned records, execution results, validation reports, events, metrics, traces, logs, and acceptance evidence.
- Human-readable status summaries for dashboards, operations, release notes, and incident review.

Public Interfaces

- `create_opportunity_intelligence(request)`
- `get_opportunity_intelligence_state(id)`
- `validate_opportunity_intelligence(payload)`
- `execute_opportunity_intelligence(command)`
- `audit_opportunity_intelligence(scope)`

Internal Interfaces

- Event Bus for durable event publication and subscription.
- Service Container for dependency injection and lifecycle management.
- Runtime Manager for execution dispatch and cancellation.
- Storage layer for records, snapshots, indexes, and evidence.
- Governance service for approval, policy, risk, and compliance checks.

Events

- `opportunity-intelligence.created`
- `opportunity-intelligence.validated`
- `opportunity-intelligence.started`
- `opportunity-intelligence.completed`
- `opportunity-intelligence.failed`
- `opportunity-intelligence.recovered`
- `opportunity-intelligence.accepted`

Storage Requirements

- Store canonical records with identifiers, owner, version, state, timestamps, policy version, and trace ID.
- Store immutable event history and mutable read models separately where practical.
- Store evidence, validation reports, and recovery metadata with retention rules.

Security Requirements

- Enforce authentication, authorization, tenant isolation, and field-level redaction.
- Protect secrets by reference only; never persist raw credentials in logs, events, Markdown, or exports.
- Require approval for privileged, destructive, externally visible, financial, or compliance-sensitive actions.

Observability Requirements

- Emit structured logs with correlation IDs and decision reasons.
- Emit metrics for request count, success rate, failure rate, latency, retries, recovery success, and acceptance failures.
- Provide traces across public interface, policy validation, runtime execution, storage, events, and callbacks.

Failure Modes

- Invalid input, missing dependency, expired permission, unavailable runtime, conflicting state, duplicate command, or failed downstream provider.
- Incomplete event delivery, partial persistence, timeout, stalled execution, or acceptance failure.

Recovery Rules

- Retry idempotent commands with bounded backoff.
- Rebuild read models from event history when projections drift.
- Escalate to governance when automatic recovery changes risk, scope, budget, deadline, or external side effects.
- Preserve failed attempts and recovery decisions as audit evidence.

Test Requirements

- Unit tests for schemas, state transitions, permission checks, and event payloads.
- Integration tests for public interfaces, storage, event bus, runtime dependencies, and governance approvals.
- Failure tests for retries, duplicate events, unavailable dependencies, and recovery paths.
- Acceptance tests proving traceability from requirement to evidence.

Acceptance Criteria

- The specification is complete enough to create implementation tickets without hidden architectural decisions.
- Interfaces, events, storage, security, observability, failures, recovery, and tests are documented.
- The document traces to Blueprint, Standards, and release artifacts.

Implementation Notes

- Prefer small versioned interfaces and append-only event history.
- Keep provider-specific details behind adapters.
- Treat auditability and recovery as first-class design constraints.
- Validate schemas before work reaches runtime execution.

Traceability

Source	Relationship
MAL-V06-OPPORTUNITY-INTELLIGENCE-SPEC-02	Defines v31 implementation requirements for Opportunity Intelligence.

Source	Relationship
Volume 03	Blueprint source.
Volume 05	Engineering standards source.
RELEASE_NOTES_v31_draft.md	Release evidence.

Version History

Version	Date	Status	Notes
v31 draft	2026-06-29	Draft	Initial specification for Opportunity Intelligence.

Freelance Automation Specification

Title: Freelance Automation Specification

Version: v31 draft

Status: Draft

Specification ID: MAL-V06-FREELANCE-AUTOMATION-SPEC-03

Related Blueprint Documents

- Volume 03 Master Blueprint
- Volume 04 Canonical Dictionary
- Volume 05 Engineering Standards
- Prior Volume 06 Specifications

Purpose

Define implementation-level requirements for Freelance Automation in Mariam AI Enterprise OS so engineering teams can build, test, operate, and govern it consistently.

Scope

- Covers system responsibilities, runtime behavior, interfaces, events, storage, security, observability, and acceptance evidence.
- Includes interactions with Enterprise Core, AI Society, Runtime Ecosystem, Knowledge System, Capability System, Governance, and Infrastructure.
- Excludes application-specific code, vendor credentials, and temporary implementation shortcuts.

Responsibilities

- Maintain authoritative state and lifecycle rules for Freelance Automation.
- Validate inputs, permissions, dependencies, and policy constraints before execution.
- Emit auditable events for lifecycle transitions, failures, recovery, and acceptance.
- Provide deterministic behavior that can be tested and traced across releases.

Functional Requirements

- SHALL expose versioned public interfaces for create, read, update, execute, validate, and audit operations.
- SHALL validate all inbound requests against schema, identity, policy, and dependency rules.
- SHALL support idempotency keys for commands that may be retried.
- SHALL publish events for accepted, rejected, started, completed, failed, recovered, and archived states.
- SHALL produce evidence bundles for review, compliance, and release acceptance.

Non-Functional Requirements

- MUST preserve tenant isolation, least privilege, and traceable authorization.
- MUST support observable latency, throughput, failure rate, queue depth, and recovery metrics.

- MUST recover safely after process restart, duplicate event delivery, or partial storage failure.
- SHOULD degrade gracefully when optional providers, plugins, connectors, or models are unavailable.

Inputs

- User, agent, workflow, runtime, provider, plugin, connector, and governance requests.
- Configuration, policies, feature flags, schemas, secrets references, and dependency metadata.
- Runtime events, task graph state, knowledge records, capability scores, and audit context.

Outputs

- Versioned records, execution results, validation reports, events, metrics, traces, logs, and acceptance evidence.
- Human-readable status summaries for dashboards, operations, release notes, and incident review.

Public Interfaces

- `create_freelance_automation(request)`
- `get_freelance_automation_state(id)`
- `validate_freelance_automation(payload)`
- `execute_freelance_automation(command)`
- `audit_freelance_automation(scope)`

Internal Interfaces

- Event Bus for durable event publication and subscription.
- Service Container for dependency injection and lifecycle management.
- Runtime Manager for execution dispatch and cancellation.
- Storage layer for records, snapshots, indexes, and evidence.
- Governance service for approval, policy, risk, and compliance checks.

Events

- `freelance-automation.created`
- `freelance-automation.validated`
- `freelance-automation.started`
- `freelance-automation.completed`
- `freelance-automation.failed`
- `freelance-automation.recovered`
- `freelance-automation.accepted`

Storage Requirements

- Store canonical records with identifiers, owner, version, state, timestamps, policy version, and trace ID.
- Store immutable event history and mutable read models separately where practical.
- Store evidence, validation reports, and recovery metadata with retention rules.

Security Requirements

- Enforce authentication, authorization, tenant isolation, and field-level redaction.
- Protect secrets by reference only; never persist raw credentials in logs, events, Markdown, or exports.
- Require approval for privileged, destructive, externally visible, financial, or compliance-sensitive actions.

Observability Requirements

- Emit structured logs with correlation IDs and decision reasons.
- Emit metrics for request count, success rate, failure rate, latency, retries, recovery success, and acceptance failures.
- Provide traces across public interface, policy validation, runtime execution, storage, events, and callbacks.

Failure Modes

- Invalid input, missing dependency, expired permission, unavailable runtime, conflicting state, duplicate command, or failed downstream provider.
- Incomplete event delivery, partial persistence, timeout, stalled execution, or acceptance failure.

Recovery Rules

- Retry idempotent commands with bounded backoff.
- Rebuild read models from event history when projections drift.
- Escalate to governance when automatic recovery changes risk, scope, budget, deadline, or external side effects.
- Preserve failed attempts and recovery decisions as audit evidence.

Test Requirements

- Unit tests for schemas, state transitions, permission checks, and event payloads.
- Integration tests for public interfaces, storage, event bus, runtime dependencies, and governance approvals.
- Failure tests for retries, duplicate events, unavailable dependencies, and recovery paths.
- Acceptance tests proving traceability from requirement to evidence.

Acceptance Criteria

- The specification is complete enough to create implementation tickets without hidden architectural decisions.
- Interfaces, events, storage, security, observability, failures, recovery, and tests are documented.
- The document traces to Blueprint, Standards, and release artifacts.

Implementation Notes

- Prefer small versioned interfaces and append-only event history.
- Keep provider-specific details behind adapters.
- Treat auditability and recovery as first-class design constraints.
- Validate schemas before work reaches runtime execution.

Traceability

Source	Relationship
MAL-V06-FREELANCE-AUTOMATION-SPEC-03	Defines v31 implementation requirements for Freelance Automation.

Source	Relationship
Volume 03	Blueprint source.
Volume 05	Engineering standards source.
RELEASE_NOTES_v31_draft.md	Release evidence.

Version History

Version	Date	Status	Notes
v31 draft	2026-06-29	Draft	Initial specification for Freelance Automation.

Remote Work Automation Specification

Title: Remote Work Automation Specification

Version: v31 draft

Status: Draft

Specification ID: MAL-V06-REMOTE-WORK-AUTOMATION-SPEC-04

Related Blueprint Documents

- Volume 03 Master Blueprint
- Volume 04 Canonical Dictionary
- Volume 05 Engineering Standards
- Prior Volume 06 Specifications

Purpose

Define implementation-level requirements for Remote Work Automation in Mariam AI Enterprise OS so engineering teams can build, test, operate, and govern it consistently.

Scope

- Covers system responsibilities, runtime behavior, interfaces, events, storage, security, observability, and acceptance evidence.
- Includes interactions with Enterprise Core, AI Society, Runtime Ecosystem, Knowledge System, Capability System, Governance, and Infrastructure.
- Excludes application-specific code, vendor credentials, and temporary implementation shortcuts.

Responsibilities

- Maintain authoritative state and lifecycle rules for Remote Work Automation.
- Validate inputs, permissions, dependencies, and policy constraints before execution.
- Emit auditable events for lifecycle transitions, failures, recovery, and acceptance.
- Provide deterministic behavior that can be tested and traced across releases.

Functional Requirements

- SHALL expose versioned public interfaces for create, read, update, execute, validate, and audit operations.
- SHALL validate all inbound requests against schema, identity, policy, and dependency rules.
- SHALL support idempotency keys for commands that may be retried.
- SHALL publish events for accepted, rejected, started, completed, failed, recovered, and archived states.
- SHALL produce evidence bundles for review, compliance, and release acceptance.

Non-Functional Requirements

- MUST preserve tenant isolation, least privilege, and traceable authorization.
- MUST support observable latency, throughput, failure rate, queue depth, and recovery metrics.

- MUST recover safely after process restart, duplicate event delivery, or partial storage failure.
- SHOULD degrade gracefully when optional providers, plugins, connectors, or models are unavailable.

Inputs

- User, agent, workflow, runtime, provider, plugin, connector, and governance requests.
- Configuration, policies, feature flags, schemas, secrets references, and dependency metadata.
- Runtime events, task graph state, knowledge records, capability scores, and audit context.

Outputs

- Versioned records, execution results, validation reports, events, metrics, traces, logs, and acceptance evidence.
- Human-readable status summaries for dashboards, operations, release notes, and incident review.

Public Interfaces

- `create_remote_work_automation(request)`
- `get_remote_work_automation_state(id)`
- `validate_remote_work_automation(payload)`
- `execute_remote_work_automation(command)`
- `audit_remote_work_automation(scope)`

Internal Interfaces

- Event Bus for durable event publication and subscription.
- Service Container for dependency injection and lifecycle management.
- Runtime Manager for execution dispatch and cancellation.
- Storage layer for records, snapshots, indexes, and evidence.
- Governance service for approval, policy, risk, and compliance checks.

Events

- `remote-work-automation.created`
- `remote-work-automation.validated`
- `remote-work-automation.started`
- `remote-work-automation.completed`
- `remote-work-automation.failed`
- `remote-work-automation.recovered`
- `remote-work-automation.accepted`

Storage Requirements

- Store canonical records with identifiers, owner, version, state, timestamps, policy version, and trace ID.
- Store immutable event history and mutable read models separately where practical.
- Store evidence, validation reports, and recovery metadata with retention rules.

Security Requirements

- Enforce authentication, authorization, tenant isolation, and field-level redaction.
- Protect secrets by reference only; never persist raw credentials in logs, events, Markdown, or exports.
- Require approval for privileged, destructive, externally visible, financial, or compliance-sensitive actions.

Observability Requirements

- Emit structured logs with correlation IDs and decision reasons.
- Emit metrics for request count, success rate, failure rate, latency, retries, recovery success, and acceptance failures.
- Provide traces across public interface, policy validation, runtime execution, storage, events, and callbacks.

Failure Modes

- Invalid input, missing dependency, expired permission, unavailable runtime, conflicting state, duplicate command, or failed downstream provider.
- Incomplete event delivery, partial persistence, timeout, stalled execution, or acceptance failure.

Recovery Rules

- Retry idempotent commands with bounded backoff.
- Rebuild read models from event history when projections drift.
- Escalate to governance when automatic recovery changes risk, scope, budget, deadline, or external side effects.
- Preserve failed attempts and recovery decisions as audit evidence.

Test Requirements

- Unit tests for schemas, state transitions, permission checks, and event payloads.
- Integration tests for public interfaces, storage, event bus, runtime dependencies, and governance approvals.
- Failure tests for retries, duplicate events, unavailable dependencies, and recovery paths.
- Acceptance tests proving traceability from requirement to evidence.

Acceptance Criteria

- The specification is complete enough to create implementation tickets without hidden architectural decisions.
- Interfaces, events, storage, security, observability, failures, recovery, and tests are documented.
- The document traces to Blueprint, Standards, and release artifacts.

Implementation Notes

- Prefer small versioned interfaces and append-only event history.
- Keep provider-specific details behind adapters.
- Treat auditability and recovery as first-class design constraints.
- Validate schemas before work reaches runtime execution.

Traceability

Source	Relationship
MAL-V06-REMOTE-WORK-AUTOMATION-SPEC-04	Defines v31 implementation requirements for Remote Work Automation.

Source	Relationship
Volume 03	Blueprint source.
Volume 05	Engineering standards source.
RELEASE_NOTES_v31_draft.md	Release evidence.

Version History

Version	Date	Status	Notes
v31 draft	2026-06-29	Draft	Initial specification for Remote Work Automation.